

ADVANCED AUTOMOTIVE ECU REPORT



1. Project Objective

The main objective of this project is to design and implement a simplified Automotive Electronic Control Unit (**ECU**) using Verilog Hardware Description Language (HDL). The project demonstrates how digital design techniques can be applied to develop a system capable of monitoring vehicle conditions, processing sensor inputs, and generating appropriate control signals for different automotive functions.

The proposed ECU combines several independent automotive subsystems into a single digital control architecture. Each subsystem is implemented as an independent Verilog module responsible for a specific vehicle function. Then these modules are integrated together in a top module. These modules are designed to operate according to predefined conditions and provide appropriate control or warning signals.

The five main modules implemented in this project are:

1. **Active Spoiler System**
2. **Advanced Parking System**
3. **Drive Mode Controller**
4. **Smart RPM Limiter**
5. **Traction and ABS System**

The project also aims to demonstrate important concepts in digital design, including:

- Synchronous and asynchronous reset operation
- Sequential and combinational logic
- Counters and timers
- Pulse Width Modulation (PWM)
- State-based control
- Sensor signal processing
- Protection and safety logic
- Real-time decision making
- Modular digital system design

Each module is developed and tested independently using a dedicated testbench. After verifying the individual modules, they can be integrated as parts of a larger automotive ECU architecture.

2. Introduction to Electronic Control Units (ECUs)

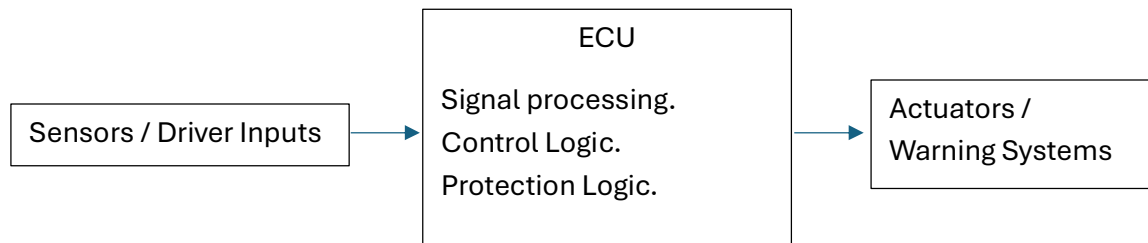
An Electronic Control Unit (**ECU**) is an embedded electronic system used in modern vehicles to monitor vehicle conditions and control different automotive functions.

A vehicle contains many sensors that continuously provide information about the current state of the vehicle. Examples of these parameters include:

- Vehicle speed
- Engine speed (RPM)
- Throttle position
- Brake pedal status
- Wheel speeds
- Distance from nearby obstacles
- Driving mode selected by the driver

The ECU receives this information as input signals, processes the signals according to predefined control algorithms, and generates output signals that control actuators or provide warnings to the driver.

The general operation of an ECU can be represented as:



In a real vehicle, the ECU is usually implemented using a microcontroller or microprocessor together with memory, communication interfaces, input conditioning circuits, and output drivers.

In this project, the ECU concept is implemented using **Verilog HDL**, allowing the control logic to be represented as digital hardware suitable for implementation on an FPGA.

3. Why Do We Need an ECU?

Modern vehicles contain a large number of electronic systems. Without centralized electronic control, each function would need to operate independently, making the vehicle more difficult to control and coordinate.

An ECU provides several important advantages.

3.1 Real-Time Monitoring

The ECU can continuously monitor vehicle parameters such as speed, RPM, wheel speed, and brake status.

For example, the RPM limiter monitors engine RPM and determines whether the engine is operating normally or approaching a critical RPM level.

3.2 Automatic Control

The ECU can automatically control vehicle systems based on sensor inputs.

For example, the Active Spoiler System automatically changes the spoiler position according to the vehicle speed and brake pedal status.

3.3 Improved Safety

Many ECU functions are designed to improve vehicle safety.

Examples include:

- ABS to help prevent wheel locking during braking
- TCS to reduce excessive wheel slip during acceleration
- Parking assistance to warn the driver about nearby obstacles
- RPM limiting to protect the engine from excessive speed

3.4 Improved Vehicle Performance

The ECU can modify vehicle behavior depending on the driver's selected mode.

For example, the Drive Mode Controller provides different throttle-to-motor responses for:

- Eco mode
- Normal mode
- Sport mode
- Track mode

This allows the same vehicle to provide different performance characteristics depending on the selected driving mode.

3.5 Integration of Multiple Systems

One of the most important advantages of an ECU is that multiple vehicle functions can be integrated into a unified electronic control architecture.

In this project, five different modules represent different automotive systems:

- Active Spoiler System
- Advanced Parking System
- Drive Mode Controller
- Smart RPM Limiter
- Traction and ABS System

The modules receive different inputs and operate independently according to their specific control logic.

4. Module 1: Active Spoiler System

The Active Spoiler System is designed to automatically control the position of a vehicle's aerodynamic spoiler according to the vehicle's speed and braking condition.

An active spoiler can have different positions depending on the driving situation.

At low speeds, the spoiler remains closed to reduce unnecessary aerodynamic drag. As the vehicle speed increases, the spoiler moves to different positions to provide greater aerodynamic stability. When the driver applies the brake pedal at a sufficiently high speed, the spoiler enters an airbrake position.

4.1 Module inputs:

- Clk: system clock
- rst: Active high reset
- vehicle_speed:
- brake_pedal:

4.2 Module Outputs:

- spoiler_pwm: PWM control signal for the spoiler actuator
- airbrake_led: Indicates that the airbrake function is active

4.3 Speed Thresholds and Spoiler Angles:

The module defines two speed thresholds and four spoiler positions:

```
localparam SPEED_CITY = 9'd80;
localparam SPEED_HIGHWAY = 9'd150;
localparam ANGLE_CLOSED = 8'd0;
localparam ANGLE_CRUISE = 8'd64;
localparam ANGLE_TRACK = 8'd128;
localparam ANGLE_AIRBRAKE = 8'd255;
```

Based on the inputs, the module determines a target spoiler angle.

Condition	Spoiler Position
Vehicle speed $\leq$ 80	Closed
$80 <$ Vehicle speed $\leq$ 150	Cruise position
Vehicle speed $>$ 150	Track position
Brake pressed and speed $>$ 80	Airbrake position

4.4 Target Angle and Airbrake Control:

The following combinational block determines the target spoiler angle:

```
always @(*) begin
    airbrake_led = 1'b0;

    if (brake_pedal == 1'b1 && vehicle_speed > SPEED_CITY)
begin
        target_angle = ANGLE_AIRBRAKE;
        airbrake_led = 1'b1;
    end
    else if (vehicle_speed > SPEED_HIGHWAY) begin
        target_angle = ANGLE_TRACK; // 0.5
    end
    else if (vehicle_speed > SPEED_CITY) begin
        target_angle = ANGLE_CRUISE; //0.25
    end
    else begin
        target_angle = ANGLE_CLOSED; //0
    end
end
```

This block is responsible for deciding the required spoiler position.

The braking condition has the highest priority. If the brake pedal is pressed and the vehicle speed is greater than the city-speed threshold, the spoiler moves toward the airbrake position and the airbrake LED is activated.

If the airbrake condition is not active, the system checks the vehicle speed.

4.5 Smooth Spoiler Movement:

The following sequential block controls the gradual movement of the spoiler:

```
always @(posedge clk or posedge rst) begin
    if (rst) begin
        current_angle <= ANGLE_CLOSED;
        spoiler_timer <= 0;
    end
    else begin
        if (spoiler_timer >= 40) begin // 40 * 255 * 10 ns
= 102 us   resonable for sim (irl shuold be >40000 )
            spoiler_timer <= 0;

            if (current_angle < target_angle)
                current_angle <= current_angle + 1;
            else if (current_angle > target_angle)
                current_angle <= current_angle - 1;
        end
        else begin
            spoiler_timer <= spoiler_timer + 1;
        end
    end
end
```

This block prevents the spoiler position from changing instantly.

The current_angle gradually approaches the target_angle.

If the current angle is smaller than the target angle, it increases by one.

If the current angle is greater than the target angle, it decreases by one.

The spoiler_timer determines how frequently the angle is updated.

This produces a simplified model of smooth mechanical actuator movement.

4.6 Servo PWM Generation:

The final block generates the PWM signal:

```
always @(posedge clk or posedge rst) begin
    if (rst) begin
        pwm_counter <= 8'd0;
        spoiler_pwm <= 1'b0;
    end else begin
        pwm_counter <= pwm_counter + 1;

        if (pwm_counter < current_angle)
            spoiler_pwm <= 1'b1;
        else
            spoiler_pwm <= 1'b0;
    end
end
```

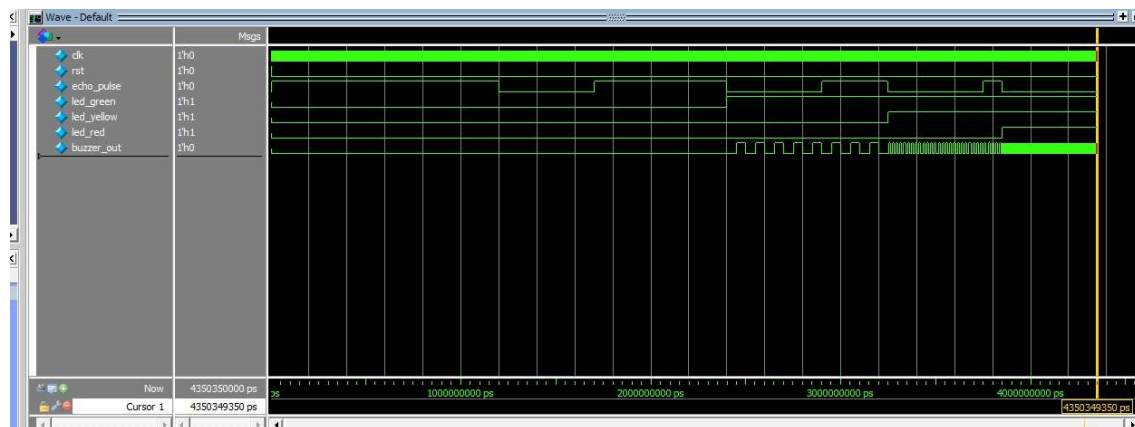
The pwm_counter continuously counts through its 8-bit range.

The PWM output is high when:

$\text{pwm_counter} < \text{current_angle}$

Therefore, increasing current_angle increases the PWM duty cycle.

4.7 Module Test Bench:



The testbench confirms the module is working as intended

5. Advanced Parking System:

6.1 Module Overview

The Advanced Parking System assists the driver when the vehicle approaches an obstacle.

The module uses an ultrasonic sensor represented by the echo_pulse input. The duration of the echo pulse is measured using a counter.

The measured pulse duration is classified into four zones:

- Far
- Mid
- Close
- Danger

The module provides:

- Visual warning using LEDs
- Audible warning using a buzzer

6.2 Module inputs:

- Clk: system clock
- rst: Active high reset
- echo_pulse: Ultrasonic echo signal

6.3 Module Outputs:

- led_green:
- led_yellow:
- led_red:
- buzzer_out:

6.4 Distance Threshold Definitions:

The distance thresholds are defined as:

```
localparam ZONE_MID      = 16'd10000;
localparam ZONE_CLOSE    = 16'd5000;
localparam ZONE_DANGER   = 16'd2000;
```

The pulse duration is classified as follows:

Pulse Width	Distance Zone
< 2000	Danger
2000–4999	Close
5000–9999	Mid
≥ 10000	Far

The distance_zone register stores the current zone.

6.5 Ultrasonic Pulse Measurement and Distance Classification

The measurement block is:

```
always @(posedge clk or posedge rst) begin
    if (rst) begin
        pulse_arr_time <= 0;
        distance_zone <= 0;
    end
    else begin
        if (echo_pulse) begin
            if (pulse_arr_time < 16'hFFFF) begin
                pulse_arr_time <= pulse_arr_time + 1;
            end
        end
        else if (pulse_arr_time > 0) begin
            if (pulse_arr_time < ZONE_DANGER)
                distance_zone <= 3;
            else if (pulse_arr_time < ZONE_CLOSE)
                distance_zone <= 2;
            else if (pulse_arr_time < ZONE_MID)
                distance_zone <= 1;
            else
                distance_zone <= 0;

            pulse_arr_time <= 0;
        end
    end
end
```

When the echo pulse is high, pulse_arr_time counts the duration of the pulse.

When the echo pulse becomes low, the measured value is compared with the predefined thresholds.

The distance zone is then selected.

The counter also includes overflow protection by preventing the value from exceeding 16'hFFFF.

6.6 LED Warning Block

```
always @(*) begin
    led_green  = (distance_zone >= 1);
    led_yellow = (distance_zone >= 2);
    led_red    = (distance_zone == 3);
end
```

The LEDs provide increasing visual warnings as the obstacle becomes closer.

The behavior is:

Far → No LEDs

Mid → Green

Close → Green + Yellow

Danger → Green + Yellow + Red

6.7 Buzzer Control Block:

The buzzer is controlled by the following sequential block:

```
always @(posedge clk or posedge rst) begin
    if (rst || distance_zone == 0) begin
        tone_timer <= 0;
        buzzer_out <= 0;
    end
    else begin
        tone_timer <= tone_timer + 1;
        case (distance_zone)
            1: begin
                if (tone_timer >= 500) begin
                    tone_timer <= 0;
                    buzzer_out <= ~buzzer_out;
                end
            end
            2: begin
                if (tone_timer >= 100) begin
                    tone_timer <= 0;
                    buzzer_out <= ~buzzer_out;
                end
            end
        endcase
    end
end
```

```

end
3: begin
    if (tone_timer >= 30) begin
        tone_timer <= 0;
        buzzer_out <= ~buzzer_out;
    end
end
endcase
end
end

```

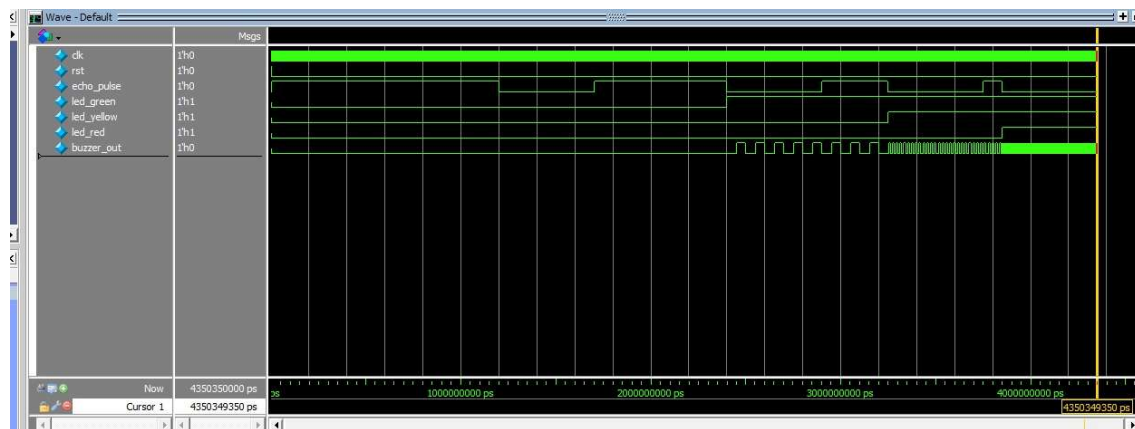
The tone_timer determines how quickly the buzzer output changes state.

The closer the obstacle becomes, the smaller the timer threshold becomes, causing the buzzer to toggle more frequently.

Therefore:

- 1) Obstacle Gets Closer
- 2) Distance Zone Increases
- 3) Buzzer Toggles Faster
- 4) More Urgent Warning

6.8 Module Test Bench:



The testbench confirms the module is working as intended

7 Drive Mode Controller:

7.1 Module Overview:

The Drive Mode Controller modifies the motor response according to the driver's selected driving mode.

The four available modes are:

- Eco
- Normal
- Sport
- Track

The controller receives throttle input and drive mode and generates a motor PWM signal.

7.2 Module inputs:

- Clk: system clock
- rst: Active high reset
- throttle_in: Throttle input
- drive_mode: Selected driving mode

7.3 Module Outputs:

- motor_pwm:

7.4 Drive Mode Mapping Block:

The target motor duty cycle is selected using the following combinational logic:

```
always @(*) begin
    case (drive_mode)
        2'b01: begin // Normal
            case (throttle_in)
                2'b00: target_duty = 8'd0;
                2'b01: target_duty = 8'd64;
                2'b10: target_duty = 8'd128;
                2'b11: target_duty = 8'd255;
            endcase
        end

        2'b00: begin // Eco
            case (throttle_in)
                2'b00: target_duty = 8'd0;
                2'b01: target_duty = 8'd32;
                2'b10: target_duty = 8'd64;
                2'b11: target_duty = 8'd128;
            endcase
        end

        2'b10: begin // Sport
            case (throttle_in)
                2'b00: target_duty = 8'd0;
                2'b01: target_duty = 8'd128;
                2'b10: target_duty = 8'd192;
                2'b11: target_duty = 8'd255;
            endcase
        end

        2'b11: begin // Track
            case (throttle_in)
                2'b00: target_duty = 8'd0;
                2'b01: target_duty = 8'd192;
                2'b10: target_duty = 8'd255;
                2'b11: target_duty = 8'd255;
            endcase
        end

        default: target_duty = 8'd0;
    endcase
end
```

This block maps the throttle input to a target PWM duty cycle according to the selected drive mode.

Eco mode limits the maximum duty cycle, while Sport and Track modes provide a more aggressive response.

7.5 Smooth Rate Limiter:

The smooth rate limiter is implemented as:

```
always @(posedge clk or posedge rst) begin
    if (rst) begin
        current_duty <= 8'd0;
        smooth_timer <= 0;
    end
    else begin
        if (smooth_timer >= 50) begin
            smooth_timer <= 0;

            if (current_duty < target_duty)
                current_duty <= current_duty + 1;
            else if (current_duty > target_duty)
                current_duty <= current_duty - 1;
        end
        else begin
            smooth_timer <= smooth_timer + 1;
        end
    end
end
```

This block prevents the motor power from immediately jumping to the target value.

Instead, current_duty gradually approaches target_duty.

The timer determines the update rate.

7.6 PWM Generation :

The motor PWM is generated by:

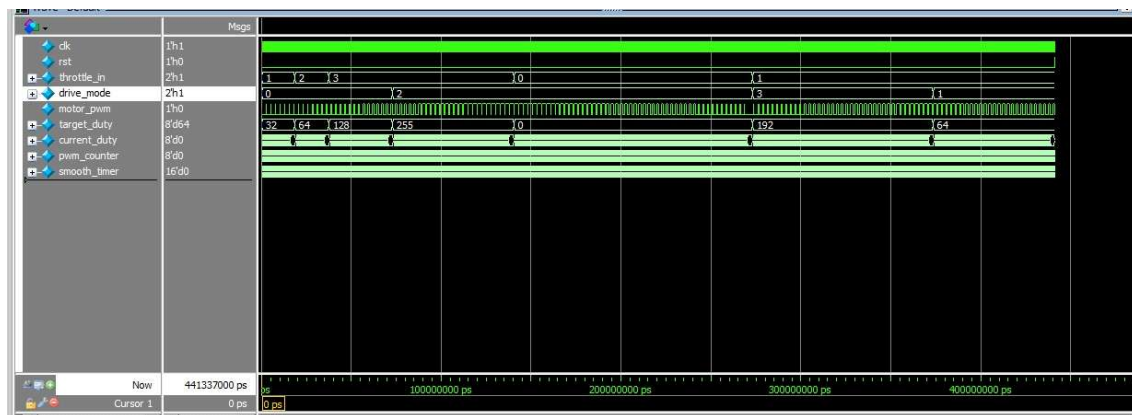
```
always @(posedge clk or posedge rst) begin
    if (rst) begin
        pwm_counter <= 8'd0;
        motor_pwm <= 1'b0;
    end
    else begin
        pwm_counter <= pwm_counter + 1;

        if (pwm_counter < current_duty)
            motor_pwm <= 1'b1;
        else
            motor_pwm <= 1'b0;
    end
end
end
```

The PWM duty cycle is controlled by current_duty.

The larger the duty cycle, the greater the average motor power represented by the PWM signal.

7.7 Module Test Bench:



The testbench confirms the module is working as intended

8. Smart RPM Limiter

8.1 Module Overview:

The Smart RPM Limiter protects the engine from excessive RPM.

It also implements a simplified launch-control function.

The module uses:

- Engine RPM
- Vehicle speed
- Brake pedal
- Throttle input

The outputs are:

- spark_retard
- fuel_cut

8.2 RPM Threshold Definitions:

The module defines:

```
localparam RPM_LAUNCH    = 4'd4;  
localparam RPM_SOFT_MAX  = 4'd9;  
localparam RPM_HARD_MAX  = 4'd10;
```

These represent the launch, soft-limit, and hard-limit RPM thresholds.

8.3 Launch Control Block:

```
if (vehicle_speed == 9'd0 &&  
    brake_pedal == 1'b1 &&  
    throttle_in == 2'b11)
```

The system therefore detects the condition where the vehicle is stationary, the brake pedal is pressed, and maximum throttle is applied.

When the engine RPM reaches the launch threshold, spark retard is activated:

```
if (engine_rpm >= RPM_LAUNCH) begin  
    spark_retard <= 1'b1;  
    fuel_cut <= 1'b0;  
end
```

This provides a simplified launch-control mechanism.

8.4 Soft RPM Limit

During normal driving, the system checks the soft RPM threshold:

```
else if (engine_rpm >= RPM_SOFT_MAX) begin
    fuel_cut <= 1'b0;
    spark_retard <= 1'b1;
end
```

When the engine reaches the soft RPM limit, spark retard is activated.

This represents the first stage of engine protection.

8.5 Hard RPM Limit:

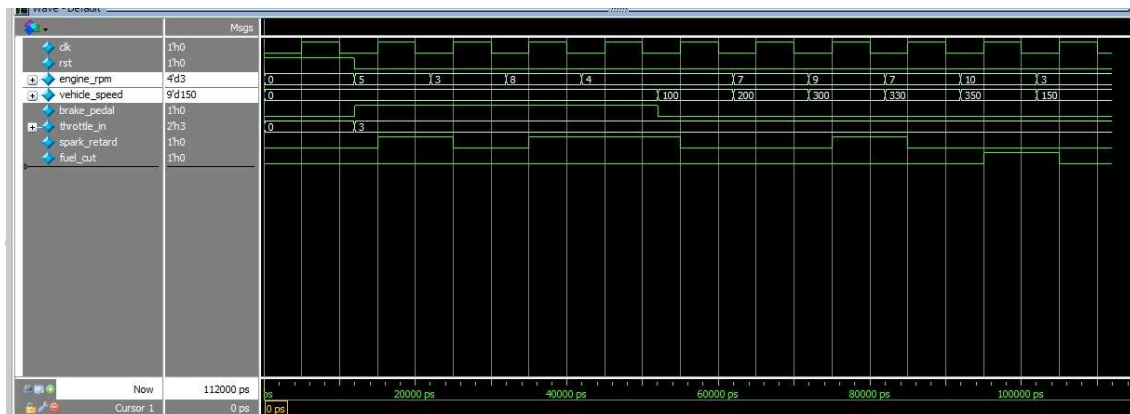
If the engine reaches the hard maximum RPM:

```
if (engine_rpm >= RPM_HARD_MAX) begin
    fuel_cut <= 1'b1;
    spark_retard <= 1'b0;
end
```

The fuel-cut output is activated.

The protection strategy can therefore be represented as:

8.6 Module Test Bench:



The testbench confirms the module is working as intended

9. Traction and ABS System:

9.1 Module Overview:

The Traction and ABS System combines two safety functions:

- Anti-lock Braking System (ABS)
- Traction Control System (TCS)

The system compares front and rear wheel speeds to detect excessive wheel-speed difference

9.2 Inputs:

- Clk: system clock
- rst: Active high reset
- front_wheel_speed:
- rear_wheel_speed:
- brake_pedal:
- ecu_motor_pwm_in:

9.3 outputs:

- final_motor_pwm:
- abs_brake_pulse
- tcs_active_led
- abs_active_led

9.4 Slip Threshold Block:

The maximum allowed wheel-speed difference is defined as:

```
localparam MAX_SLIP = 8'd15;
```

The system uses this value to determine whether excessive wheel slip has occurred.

9.5 ABS Control Block:

When the brake pedal is pressed, the system enters the ABS section:

```
if (brake_pedal == 1'b1) begin
    final_motor_pwm <= 8'd0;
    tcs_active_led  <= 1'b0;

    if (front_wheel_speed > rear_wheel_speed &&
        (front_wheel_speed - rear_wheel_speed) > MAX_SLIP)
begin
    abs_brake_pulse <= ~abs_brake_pulse;
    abs_active_led  <= 1'b1;

end else begin

    abs_brake_pulse <= 1'b1;
    abs_active_led  <= 1'b0;

end
end
```

During braking, the motor PWM is set to zero.

The system compares the front and rear wheel speeds. If the speed difference exceeds the defined slip threshold, the ABS system becomes active.

The ABS indicator is also activated.

9.6 Traction Control Block:

When the brake pedal is not pressed, the system operates in traction-control mode:

```
else begin
    abs_brake_pulse <= 1'b0;
    abs_active_led  <= 1'b0;

    if (rear_wheel_speed > front_wheel_speed &&
        (rear_wheel_speed - front_wheel_speed) > MAX_SLIP)
begin
    final_motor_pwm <= (ecu_motor_pwm_in >> 1);
    tcs_active_led  <= 1'b1;

end else begin

    final_motor_pwm <= ecu_motor_pwm_in;
    tcs_active_led  <= 1'b0;

end
end
```

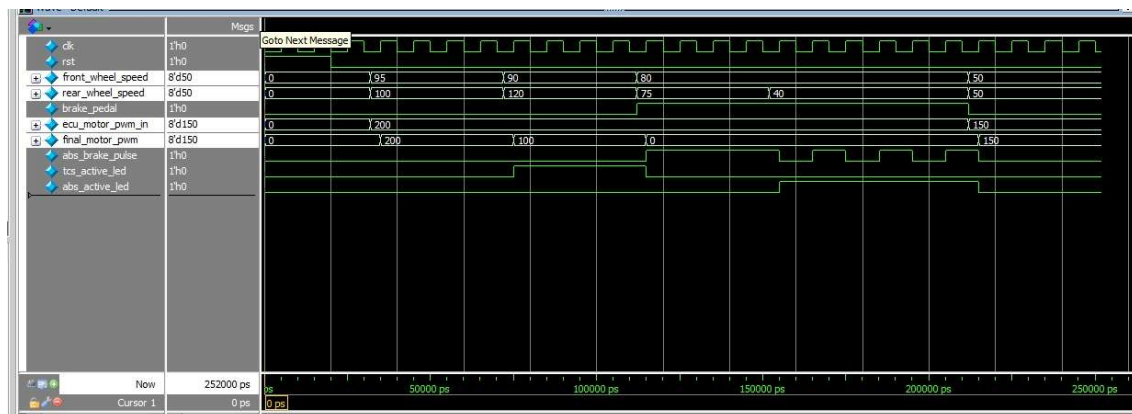
If the rear wheel speed becomes significantly greater than the front wheel speed, excessive wheel slip is detected.

The system then reduces the motor PWM by shifting the input PWM to the right by one bit.

This effectively reduces the PWM command by approximately half.

If no excessive slip is detected, the original motor PWM is passed directly to the output.

9.7 Module Test Bench:



The testbench confirms the module is working as intended